CS 141 - Lab 2: Control Structures and Arrays

Conditional Statements, Loops, and Arrays in C

Table of Contents

1. Lab Objectives
2. Exercise 1: Conditional Statements (if, if-else)
3. Exercise 2: Nested Conditionals and Logical Operators
4. Exercise 3: Switch Statement
5. Exercise 4: For Loops
6. Exercise 5: While and Do-While Loops
7. Exercise 6: Introduction to Arrays
8. Exercise 7: Array Operations with Loops
9. Challenge Exercise

Lab Objectives

By the end of this lab, you should be able to:

- Write and use conditional statements (if, if-else, nested if)
- Understand and use logical operators (&&, ||, !)
- Implement switch statements for multi-way decisions
- Write and use different types of loops (for, while, do-while)
- Understand loop control (break, continue)
- Declare and initialize arrays
- Access and manipulate array elements
- Use loops to process arrays
- Combine control structures with arrays to solve problems

Exercise 1: Conditional Statements (if, if-else)

Objective: Write programs using basic conditional statements to make decisions.

Task 1.1: Simple if Statement

1. Create a program named exercise1_1.c
2. Write a program that:
   - Reads an integer from the user (representing age)
   - If the age is 18 or older, display "You are eligible to vote."
   - Compile and test your program with different ages

Task 1.2: if-else Statement

1. Create a program named exercise1_2.c
2. Write a program that:
   - Reads a number from the user
   - If the number is positive, display "The number is positive."
   - Otherwise, display "The number is not positive."
   - Test with positive, negative, and zero values

Task 1.3: if-else if-else Chain

1. Create a program named exercise1_3.c
2. Write a program that:
   - Reads a student's score (0-100)
   - Displays the corresponding grade:
     - 90-100: "Grade: A"
     - 80-89: "Grade: B"
     - 70-79: "Grade: C"
     - 60-69: "Grade: D"
     - Below 60: "Grade: F"
   - Test with various scores

Expected Output Examples:

```
Enter your age: 20
You are eligible to vote.
```

```
Enter a number: -5
The number is not positive.
```

```
Enter your score: 85
Grade: B
```

Hints:

- Use scanf() to read input
- Use if, else if, and else keywords
- Remember to use & operator with scanf()

Exercise 2: Nested Conditionals and Logical Operators

Objective: Use nested conditionals and logical operators to solve complex decision-making problems.

Task 2.1: Nested if Statements

1. Create a program named exercise2_1.c
2. Write a program that:
   - Reads two integers from the user
   - Determines which number is larger, or if they are equal
   - Displays an appropriate message
   - Use nested if statements

Task 2.2: Logical Operators

1. Create a program named exercise2_2.c
2. Write a program that:
   - Reads a year from the user
   - Determines if it is a leap year
   - A year is a leap year if:
     - It is divisible by 4, AND
     - It is NOT divisible by 100, OR
     - It is divisible by 400
   - Display "Leap year" or "Not a leap year"

Task 2.3: Complex Condition

1. Create a program named exercise2_3.c
2. Write a program that:
   - Reads three integers (representing sides of a triangle)
   - Determines if they can form a valid triangle
   - A triangle is valid if the sum of any two sides is greater than the third side
   - Display "Valid triangle" or "Invalid triangle"

Expected Output Examples:

```
Enter first number: 10
Enter second number: 5
10 is larger than 5
```

```
Enter a year: 2024
Leap year
```

```
Enter side 1: 3
Enter side 2: 4
Enter side 3: 5
Valid triangle
```

Hints:

- Use && for AND, || for OR, ! for NOT
- Use parentheses to group conditions: (condition1 && condition2) || condition3
- Be careful with operator precedence

Exercise 3: Switch Statement

Objective: Use switch statements for multi-way decisions based on a single variable.

Task 3.1: Basic Switch

1. Create a program named exercise3_1.c
2. Write a program that:
   - Reads a single character from the user (A, B, C, D, or F)
   - Uses a switch statement to display:
     - 'A': "Excellent!"
     - 'B': "Good job!"
     - 'C': "Average"
     - 'D': "Below average"
     - 'F': "Failed"
     - Default: "Invalid grade"
   - Test with different characters

Task 3.2: Switch with Numbers

1. Create a program named exercise3_2.c
2. Write a program that:
   - Displays a menu:

     1. Addition
     2. Subtraction
     3. Multiplication
     4. Division

   - Reads the user's choice (1-4)
   - Reads two numbers
   - Uses a switch statement to perform the selected operation
   - Displays the result

Expected Output Examples:

```
Enter a grade (A-F): B
Good job!
```

```
Menu:
1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter your choice: 3
Enter first number: 5
Enter second number: 4
Result: 20
```

Hints:

- Use switch(variable) { case value: ... break; }
- Don't forget break; statements
- Include a default: case
- Characters in switch: case 'A':

Exercise 4: For Loops

Objective: Understand and use for loops for repetitive tasks.

Task 4.1: Basic For Loop

1. Create a program named exercise4_1.c
2. Write a program that:
   - Uses a for loop to display numbers from 1 to 10
   - Each number on a separate line

Task 4.2: For Loop with User Input

1. Create a program named exercise4_2.c
2. Write a program that:
   - Reads a positive integer n from the user
   - Uses a for loop to display all numbers from 1 to n
   - Displays the sum of all numbers from 1 to n

Task 4.3: Nested For Loops

1. Create a program named exercise4_3.c
2. Write a program that:
   - Uses nested for loops to display a pattern:

     ```
     *
     **
     ***
     ****
     *****
     ```

   - The number of rows should be read from the user

Task 4.4: For Loop with Conditions

1. Create a program named exercise4_4.c
2. Write a program that:
   - Uses a for loop to find and display all even numbers between 1 and 50
   - Display 5 numbers per line

Expected Output Examples:

```
1
2
3
...
10
```

```
Enter n: 5
1 2 3 4 5
Sum: 15
```

```
Enter number of rows: 5
*
**
***
****
*****
```

```
2 4 6 8 10

12 14 16 18 20

…

42 44 46 48 50
```

Hints:

- For loop syntax: for(initialization; condition; increment) { }
- Use % operator to check for even numbers

Exercise 5: While and Do-While Loops

Objective: Use while and do-while loops for repetitive tasks with different control structures.

Task 5.1: While Loop

1. Create a program named exercise5_1.c
2. Write a program that:
   - Reads positive integers from the user
   - Continues reading until the user enters 0
   - Counts and displays how many numbers were entered (excluding 0)
   - Displays the sum of all entered numbers

Task 5.2: Do-While Loop

1. Create a program named exercise5_2.c
2. Write a program that:
   - Uses a do-while loop to display a menu
   - Continues displaying the menu until the user chooses to exit
   - Menu options:

     1. Display Hello
     2. Display World
     3. Exit

Task 5.3: Loop Control (break and continue)

1. Create a program named exercise5_3.c
2. Write a program that:
   - Uses a while loop to read numbers from the user
   - If the number is negative, skip it (use continue)
   - If the number is 0, exit the loop (use break)
   - Otherwise, display the square of the number
   - Continue until 0 is entered

Task 5.4: Finding Maximum

1. Create a program named exercise5_4.c
2. Write a program that:
   - Reads integers from the user until -1 is entered
   - Finds and displays the maximum number entered
   - Displays how many numbers were entered

Expected Output Examples:

```
Enter numbers (0 to stop): 5 10 3 0
Count: 3
Sum: 18
```

```
Menu:
1. Display Hello
2. Display World
3. Exit
Enter choice: 1
Hello
Menu:
```

```
1. Display Hello
2. Display World
3. Exit
Enter choice: 3
Goodbye!
```

```
Enter numbers (0 to stop): 5 -2 10 3 0
5 squared = 25
10 squared = 100
3 squared = 9
```

```
Enter numbers (-1 to stop): 10 5 20 15 -1
Maximum: 20
Numbers entered: 4
```

Hints:

- While loop: while(condition) { }
- Do-while loop: do { } while(condition);
- break; exits the loop immediately
- continue; skips to the next iteration

Exercise 6: Introduction to Arrays

Objective: Understand array declaration, initialization, and basic operations.

Task 6.1: Array Declaration and Initialization

1. Create a program named exercise6_1.c
2. Write a program that:
   - Declares an array of 5 integers
   - Initializes it with values: {10, 20, 30, 40, 50}
   - Displays all array elements using a loop
   - Displays each element with its index

Task 6.2: Reading Array from User

1. Create a program named exercise6_2.c
2. Write a program that:
   - Declares an array of 10 integers

- Reads 10 numbers from the user and stores them in the array
- Displays the array in reverse order

Task 6.3: Array Operations

1. Create a program named exercise6_3.c
2. Write a program that:
   - Declares an array of 8 integers
   - Reads 8 numbers from the user
   - Finds and displays:
     - The sum of all elements
     - The average of all elements
     - The maximum element
     - The minimum element

Expected Output Examples:

```
Array elements:
Index 0: 10
Index 1: 20
Index 2: 30
Index 3: 40
Index 4: 50
```

```
Enter 10 numbers: 1 2 3 4 5 6 7 8 9 10
Array in reverse: 10 9 8 7 6 5 4 3 2 1
```

```
Enter 8 numbers: 5 10 3 8 12 7 15 2
Sum: 62
Average: 7.75
Maximum: 15
Minimum: 2
```

Hints:

- Array declaration: `int arr[5];`
- Array initialization: `int arr[5] = {1, 2, 3, 4, 5};`
- Array indexing starts at 0: `arr[0]` is the first element
- Use loops to access array elements: `for(i = 0; i < size; i++)`

Exercise 7: Array Operations with Loops

Objective: Combine arrays with loops to perform various operations.

Task 7.1: Searching in Array

1. Create a program named exercise7_1.c
2. Write a program that:
   - Declares an array of 10 integers
   - Reads 10 numbers from the user
   - Reads a number to search for
   - Searches for the number in the array
   - Displays the index if found, or "Not found" if not found

Task 7.2: Counting Elements

1. Create a program named exercise7_2.c
2. Write a program that:
   - Declares an array of 15 integers
   - Reads 15 numbers from the user
   - Counts and displays:
     - How many positive numbers
     - How many negative numbers
     - How many zeros

Task 7.3: Array Manipulation

1. Create a program named exercise7_3.c
2. Write a program that:
   - Declares an array of 12 integers
   - Reads 12 numbers from the user
   - Creates a new array containing only even numbers from the original array
   - Displays the new array

Task 7.4: Array Reversal

1. Create a program named exercise7_4.c
2. Write a program that:
   - Declares an array of 10 integers
   - Reads 10 numbers from the user
   - Reverses the array in place (without using a second array)
   - Displays the reversed array

Expected Output Examples:

```
Enter 10 numbers: 5 10 3 8 12 7 15 2 9 4
Enter number to search: 8
Found at index: 3
```

```
Enter 15 numbers: 5 -3 0 10 -5 0 7 -2 0 12 -8 3 -1 0 9
Positive: 6
Negative: 5
Zeros: 4
```

```
Enter 12 numbers: 1 2 3 4 5 6 7 8 9 10 11 12
Even numbers: 2 4 6 8 10 12
```

```
Enter 10 numbers: 1 2 3 4 5 6 7 8 9 10
Reversed array: 10 9 8 7 6 5 4 3 2 1
```

Hints:

- Use loops to iterate through arrays
- Use conditional statements to filter or search
- Use a temporary variable for swapping: `temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;`

Challenge Exercise

Objective: Combine all concepts learned to create a comprehensive program.

Task: Student Grade Management System

1. Create a program named `challenge.c`
2. Write a program that:
   - Reads the number of students (maximum 50)
   - For each student, reads their name and 3 test scores
   - Stores student data in arrays (names array and scores 2D array)
   - Calculates the average for each student
   - Determines the letter grade for each student (A, B, C, D, F)

- Displays a menu:

  1. Display all students and grades
  2. Find student with highest average
  3. Find student with lowest average
  4. Display class average
  5. Exit

- Uses a switch statement to handle menu choices

- Uses loops to process the menu until user chooses exit

Requirements:

- Use arrays to store student names and scores
- Use loops to read and process data
- Use conditional statements for grade calculation
- Use switch for menu handling
- Handle edge cases (no students, invalid input)

Expected Output Format:

```
Enter number of students: 3
Enter student 1 name: Ahmed
Enter 3 test scores: 85 90 88
Enter student 2 name: Sarra
Enter 3 test scores: 70 75 72
Enter student 3 name: Amine
Enter 3 test scores: 95 92 98

Menu:
1. Display all students and grades
2. Find student with highest average
3. Find student with lowest average
4. Display class average
5. Exit
Enter choice: 1

Student: Ahmed, Average: 87.67, Grade: B
Student: Sarra, Average: 72.33, Grade: C
Student: Amine, Average: 95.00, Grade: A
```

Hints:

- Use a 2D array for scores: `int scores[50][3];`
- Use a 2D character array for names: `char names[50][50];`

Submission Guidelines

1. Save all your programs in the week2 folder
2. Name your files exactly as specified: exercise1_1.c, exercise1_2.c, etc.
3. Make sure all programs compile without errors
4. Test each program with the provided sample inputs
5. Submit all files before the deadline

Additional Resources

- Control Flow in C:
  - if, else if, else - conditional execution
  - switch, case, default - multi-way selection
  - for - counted loops
  - while - condition-checked loops
  - do-while - condition-checked loops (executes at least once)
  - break - exit loop immediately
  - continue - skip to next iteration
- Logical Operators:
  - && - logical AND
  - || - logical OR
  - ! - logical NOT
- Arrays:
  - Declaration: type arrayName[size];
  - Initialization: int arr[5] = {1, 2, 3, 4, 5};
  - Access: arrayName[index] (index starts at 0)
  - Size: Use sizeof(arr) / sizeof(arr[0]) or a constant